



BITS Pilani
DIGITAL
Excellence made yours

Containerization Fundamentals

Docker · ECR · Multi-stage builds
GPU containers · Container security for AI

- Topic 1: Why Containers for AI/ML
- Topic 2: Docker Fundamentals
- Topic 3: Containers for AI Workloads
- Topic 4: Hands-on Setup
- Topic 5: Lesson Summary

Lesson Overview



BITS Pilani
DIGITAL
Excellence made yours

Lesson 4 · Containerization Fundamentals

5

Topics

Core areas

15

Concept Blocks

Short explainers

~65

Duration

Total minutes

FON

Level

Foundational

LEARNING OBJECTIVES

1 Understand why containers are critical for AI/ML portability and reproducibility

2 Build, run, and manage Docker containers for AI workloads

3 Handle AI-specific container challenges: GPU support, multi-stage builds, security



TOPIC 0

Lesson Introduction

What to Expect This Week

What to Expect This Week



BITS Pilani
DIGITAL
Excellence made yours

From the portability problem to production-ready GPU containers

Why Containers

The portability problem — why identical code behaves differently across machines, and how containers solve it

Docker Fundamentals

Images, layers, Dockerfile syntax, build/run/debug lifecycle, networking, volumes, and optimisation

AI-Specific Challenges

CUDA/GPU drivers, Python dependency hell, multi-stage builds, Docker Compose for AI stacks

Container Security

Non-root users, secrets management, ECR scanning, read-only filesystems, immutable image tags



TOPIC 1

01 Why Containers for AI/ML

The portability problem — and how containers solve it completely

The Works on My Machine Problem



Why AI/ML environments are uniquely hard to reproduce

Without Containers

- Python version differs between dev and prod
- pip install gives different packages each run
- CUDA version on your machine differs from the server
- One team member breaks another's environment
- Training results differ between machines silently

With Containers

- Identical Python, pip, and CUDA versions everywhere
- requirements.txt pinned → same packages every build
- Image built once, runs identically on any host with Docker
- Full environment reproducible years later from Dockerfile
- CI/CD builds the same image for dev, staging, and prod

Container Ecosystem — Docker, Podman, and Containerd



Three runtimes, one OCI standard — images built with any tool run on any runtime



Docker

Industry Standard

Most widely used container runtime. Daemon-based. Full toolchain: build, push, run, compose. Default choice for most AI/ML teams. Install via Docker Desktop or docker-ce.



Podman

Daemonless Alternative

Rootless and daemonless — more secure than Docker by default. Drop-in replacement for docker commands. Growing enterprise adoption, especially in regulated environments.



Containerd

Runtime Standard

Low-level runtime used by Kubernetes. Docker and Podman use containerd under the hood. You rarely interact with it directly, but every K8s cluster uses it.

All three implement the OCI standard — an image built with docker build runs identically on podman, containerd, or Kubernetes without modification.

Containers vs Virtual Machines — Key Differences Explained



Containers share the host OS kernel — VMs emulate complete hardware

Dimension	Virtual Machine	Container
Startup time	1–5 minutes (full OS boot)	Milliseconds to seconds
Size	10–50 GB (full OS + app)	50 MB – 2 GB (app + deps only)
Isolation	Full hardware isolation — strongest	Process isolation — sufficient for most ML
OS kernel	Each VM has its own kernel	Shared host kernel — more efficient
GPU access	Via hypervisor (complex)	Direct via NVIDIA Container Toolkit
Networking	Per VM NAT/bridging	Port mapping: -p 8080:8080
Data persistence	VM disk persists by default	Ephemeral — use named volumes
ML use case	Isolated full environments (rare)	Training jobs, serving APIs, CI/CD

How Containers Enable Scalable AI Deployments

From a single Dockerfile to auto-scaling Kubernetes inference clusters



Immutable Deployments

Build once, deploy anywhere. Same container bits run in dev, staging, and prod — no environment drift.



Horizontal Scaling

Launch 10 identical serving containers behind a load balancer — same image, different ports.



Zero-Downtime Updates

Rolling updates: new containers come up before old ones go down. No serving interruption.



Dependency Isolation

Two models needing different PyTorch versions run in separate containers on the same host.



Fast CI/CD

Layer caching means only changed layers rebuild. Container redeploys in seconds, not minutes.



Cloud-Agnostic

Same image runs on AWS ECS, GKE, Azure ACI, or on-prem K8s — no vendor lock-in.



TOPIC 2

02 Docker Fundamentals

Images, layers, Dockerfiles, networking, and volumes

Docker Architecture — Images, Containers, Registries, Layers

Six concepts that underpin everything you do with Docker



Image

Read-only filesystem snapshot built from a Dockerfile. Each instruction adds a layer. Immutable once built.



Container

A running instance of an image. Writable layer on top of the read-only image layers. Ephemeral by default.



Layer

Each RUN, COPY, or ADD instruction creates a new layer. Docker caches unchanged layers on next build.



Registry

Stores and distributes images. Docker Hub is public. Amazon ECR is private and integrates with IAM.



Docker Daemon

Background service (dockerd) managing images, containers, networks, and volumes on the host.



Docker CLI

The docker command. Sends API requests to the daemon. build, run, push, ps, exec, logs.

Dockerfile Syntax — FROM, RUN, COPY, CMD and Best Practices



The eight instructions you use in 95% of AI container builds

```
FROM python:3.10-slim
# Always pin the tag - never :latest

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1

WORKDIR /app

# COPY requirements BEFORE source code
# Docker cache: pip skipped if unchanged
COPY requirements.txt .
RUN pip install -r requirements.txt

# Source code after deps
COPY src/ ./src/

# Non-root user for security
RUN useradd -m -u 1000 mluser \
    && chown -R mluser:mluser /app
USER mluser

EXPOSE 8080
CMD ["python", "train.py"]
```

COPY vs ADD

Always use COPY — it is explicit and predictable. ADD auto-extracts tar files, which is usually unwanted.

RUN layer merging

One RUN = one layer. Merge: RUN apt-get install gcc && rm -rf /var/lib/apt/lists/*

CMD vs ENTRYPOINT

ENTRYPOINT sets the executable. CMD sets default args. Use both when you need to override just the script.

ARG vs ENV

ARG is build-time only. ENV persists into the running container. Never put secrets in either.

The Container Lifecycle — Build, Run, and Debug Commands



Ten commands that cover the complete Docker workflow

Build	<code>docker build -t loan-risk:v1 .</code> Build image from Dockerfile. -t adds a tag.
Build	<code>docker build --no-cache -t loan-risk:v1 .</code> Force full rebuild — ignores all cached layers.
Run	<code>docker run -p 8080:8080 loan-risk:v1</code> Run container, map host port to container port.
Run	<code>docker run -d --name server loan-risk:v1</code> -d = detached. --name for easy reference.
Inspect	<code>docker ps</code> List all running containers.
Inspect	<code>docker logs -f server</code> Stream logs. -f follows like tail -f.
Debug	<code>docker exec -it server bash</code> Open shell inside running container for debugging.
Debug	<code>docker inspect server</code> Full JSON metadata: ports, volumes, env vars.
Cleanup	<code>docker stop server && docker rm server</code> Stop then remove a container.
Cleanup	<code>docker system prune -af</code> Remove all stopped containers, unused images, networks.

Container Networking Basics for AI Services

Three network types and how containers communicate



Bridge Network

Default mode. Containers on the same bridge network reach each other by name.

```
$ docker network create ml-net
$ docker run --network ml-net trainer
$ docker run --network ml-net server
# server can call trainer:8080
```

Used automatically by
docker-compose for all services.



Host Network

Container shares the host's network stack — no port mapping.

```
$ docker run --network host serve.py
# Directly on host port 8080
```

Fastest for GPU inference.

⚠ Not available on macOS or
Windows.



Port Mapping

Expose a container port to the host.

```
$ docker run -p 8080:8080 server
# host:8080 → container:8080
```

In docker-compose:

ports:

- '8080:8080'

Map a range:

```
$ docker run -p 8080-8090:8080-8090
```

Image Optimisation — Reducing Size and Build Time



BITS Pilani
DIGITAL
Excellence made yours

Five techniques to cut your AI image from 4 GB to under 600 MB

Use -slim base images

python:3.10-slim saves ~700 MB vs python:3.10. python:3.10-alpine saves ~900 MB but may break scipy/numpy (musl libc).

Merge RUN commands

Each RUN = one layer. Merge: apt-get install gcc && rm -rf /var/lib/apt/lists/* Saves 100–300 MB per RUN.

Multi-stage builds

Builder stage compiles wheels. Runtime stage copies packages only. Result: 60–80% smaller image with no build tools.

Use .dockerignore

Exclude tests/, data/, *.ipynb, .git/ from build context. Smaller context = faster build and faster push to ECR.

Pin requirements.txt

Pinned versions are cached by Docker. If requirements.txt doesn't change, pip install is entirely skipped.

Remove build tools at runtime

gcc adds 200+ MB. Install it in the builder stage only. Runtime stage gets only the compiled wheels.

Docker Volumes — Persisting Data Across Container Restarts



Containers are ephemeral — volumes keep your models and datasets alive



Named Volumes (recommended for ML)

Managed by Docker. Persist after container stop/rm.

```
$ docker volume create model-store  
$ docker run -v model-store:/app/models trainer
```

```
# docker-compose.yml:  
volumes:  
  model-store:  
    name: model-store
```

```
services:  
  trainer:  
    volumes:  
      - model-store:/app/models  
  server:  
    volumes:  
      - model-store:/app/models:ro # read-only
```



Bind Mounts (for development only)

Maps a host directory into the container.
Perfect for iterating on training scripts without rebuilding.

```
$ docker run \  
-v $(pwd)/data:/app/data:ro \  
-v $(pwd)/src:/app/src \  
trainer python train.py
```

Changes to /src on host appear instantly inside the container
— no rebuild needed.

Never use bind mounts in production — the host path may not exist on another machine.



TOPIC 3

Containers for AI Workloads

GPU drivers, multi-stage builds, Docker Compose, and container security

Environment Abstraction — Python Dependencies, CUDA, and GPU Drivers



BITS Pilani
DIGITAL
Excellence made yours

The four-layer stack that must align for GPU containers to work

GPU Hardware + NVIDIA Driver

Installed on the HOST machine — not in the container. Must match or exceed the CUDA version in the image.

CUDA Toolkit (in container)

nvidia/cuda base image provides the CUDA runtime. Image CUDA version must be $\leq$ host driver version.

cuDNN (in container)

Deep learning primitives used by PyTorch and TensorFlow. Specified as cudnn8 in the base image tag.

PyTorch / TensorFlow (in container)

pip install torch==2.2.0+cu118 — the +cu118 suffix must match the CUDA version in the base image.

```
Verify on the host before building: $ nvidia-smi | $ nvcc --version | $ python -c "import torch; print(torch.cuda.is_available())"
```

Multi-Stage Builds for Optimised AI Images

Builder vs runtime — keeping only what the container needs

```
# Stage 1: Builder (has gcc, compiles wheels)
FROM python:3.10 AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip install --no-cache-dir \\\
    --target=/build/packages \\\
    -r requirements.txt

# Stage 2: Runtime (slim — no gcc, no build tools)
FROM python:3.10-slim AS runtime

# Copy ONLY compiled packages from builder stage
COPY --from=builder /build/packages \\\
    /usr/local/lib/python3.10/site-packages/

WORKDIR /app
COPY src/ ./src/

RUN useradd -m -u 1000 mluser \\\
    && chown -R mluser:mluser /app
USER mluser
EXPOSE 8080
CMD ["uvicorn", "src.api:app", "--host", "0.0.0.0"]
```



Size Comparison

Before multi-stage:

python:3.10 + gcc + numpy + torch
≈ 3.2 GB

After multi-stage:

python:3.10-slim + compiled wheels
≈ 650 MB

→ 80% reduction in image size

Production benefits:

- Faster ECR push and pull
- Smaller attack surface (no gcc in prod)
- Lower egress costs
- Faster cold starts in ECS / Lambda
- Less ECR storage cost

Docker Compose for Multi-Container AI Workflows



Define the full AI stack in one YAML file — one command to run it all

```
version: "3.9"
services:
  trainer:
    build: {dockerfile: Dockerfile.train}
    volumes:
      - ./data:/app/data:ro
      - model-store:/app/models
    networks: [ml-network]

  server:
    build:
      dockerfile: Dockerfile.serve
      target: runtime
    ports: ["8080:8080"]
    depends_on: [redis]
    healthcheck:
      test: ["CMD", "curl", "-f",
            "http://localhost:8080/health"]
      interval: 30s
      timeout: 10s
      retries: 3
    networks: [ml-network]

  redis:
    image: redis:7-alpine
    networks: [ml-network]
```

```
networks: {ml-network: {driver: bridge}}
```

```
volumes: {model-store: {}}
```

`docker compose up -d`

Start all services in background. Builds images if not yet built.

`docker compose logs -f`

Stream logs from all services simultaneously.

`docker compose down`

Stop and remove containers and networks. Named volumes survive.

`docker compose down -v`

Also removes named volumes — resets the model store.

`depends_on` vs `healthcheck`

`depends_on` waits for the container to START, not for the service to be READY. Use `healthcheck` condition: `service_healthy` for readiness.

Container Security Best Practices for ML Workloads

Six rules that prevent the most common container security mistakes



Non-Root User

RUN `useradd -m -u 1000 mluser && USER mluser`. A container breakout as root = host root access.



No Secrets in Images

Never put API keys or AWS credentials in ENV or COPY. Use AWS Secrets Manager or Docker secrets at runtime.



Read-Only Filesystem

Mount models as `:ro`. Run with `--read-only` and `--tmpfs /tmp` for temp writes.



Image Scanning

Enable ECR scan-on-push. Scans identify OS and Python library CVEs. Block deployment on critical findings.



Minimal Base Images

`python:3.10-slim` or `distroless` instead of `ubuntu`. Fewer packages = smaller attack surface.



Immutable Tags

Never `:latest` in production. Pin `image:tag` or `image@sha256:digest` so deployments are fully reproducible.



TOPIC 4

Hands-on Tooling Setup

Installing Docker and configuring ECR for AI workloads

Installing Docker and Configuring ECR for AI Workloads



BITS Pilani
DIGITAL
Excellence made yours

Four steps from zero to a fully configured AI container environment

1 Install Docker

```
# macOS / Windows: docker.com/get-started
# Ubuntu:
sudo apt-get update
sudo apt-get install docker-ce docker-ce-cli \
  containerd.io
sudo usermod -aG docker $USER
```

2 Install NVIDIA Container Toolkit (GPU)

```
distribution=$(. /etc/os-release;echo $ID$VERSION_ID)
curl -s -L
https://nvidia.github.io/nvidia-docker/gpgkey \
  | sudo apt-key add -
sudo apt-get install -y nvidia-container-toolkit
sudo systemctl restart docker
```

3 Authenticate with Amazon ECR

```
# Replace <ACCOUNT-ID> and <REGION>
aws ecr get-login-password --region <REGION> | \
docker login --username AWS --password-stdin \
  <ACCOUNT-ID>.dkr.ecr.<REGION>.amazonaws.com

# Or use: aws configure sso
```

4 Build, Push, and Verify

```
# Build and tag
docker build -t my-model:v1 -f Dockerfile.train .
docker tag my-model:v1 \

<ACCOUNT-ID>.dkr.ecr.<REGION>.amazonaws.com/my-model:v1
# Push to ECR
docker push \

<ACCOUNT-ID>.dkr.ecr.<REGION>.amazonaws.com/my-model:v1
```



TOPIC 5

05 Lesson Summary

Week 4 recap and key takeaways

Week 4 Recap — Containerization Fundamentals



Four production principles to take forward

1

Containers Solve the Reproducibility Problem

Every AI model depends on a specific combination of Python, pip packages, CUDA, and OS libraries. A container image freezes all of them at build time.

2

Layer Caching Is Your Build-Time Superpower

COPY requirements.txt and pip install BEFORE COPY of source code. Unchanged deps = pip skipped = 60-second builds instead of 10-minute builds.

3

Multi-Stage Builds Are Non-Negotiable for Production

Never ship a 3 GB image with gcc and build tools into production. Builder stage compiles; runtime stage serves — 80% smaller.

4

Security Is Not Optional

Non-root user, no secrets in ENV, ECR scan-on-push, read-only model mounts, and pinned image tags. Five minutes to add — critical to have.